Яндекс

Новости со встречи международного комитета C++





Антон Полухин

Техплатформа Городских сервисов Яндекса, руководитель группы разработки общих компонентов

Кто я?



Антон Полухин

Эксперт разработчик C++,
Группа разработки фреймворка userver,
ISO C++ National Body

- Разрабатываю 🐙 userver
- Представляю Россию в ISO
- Boost.PFR, std::stacktrace, Boost.LexicalCast, Boost.Any, Boost.TypeIndex, Boost.Conversion, Boost.DLL, Boost.Variant

Содержание

01 C++26

02 UB & IFNDR

03 operator++(int) = default;

04 [u]intptr_t

05 Contracts + virtual

06 Template names pack indexing

07 std::constant_wrapper - string

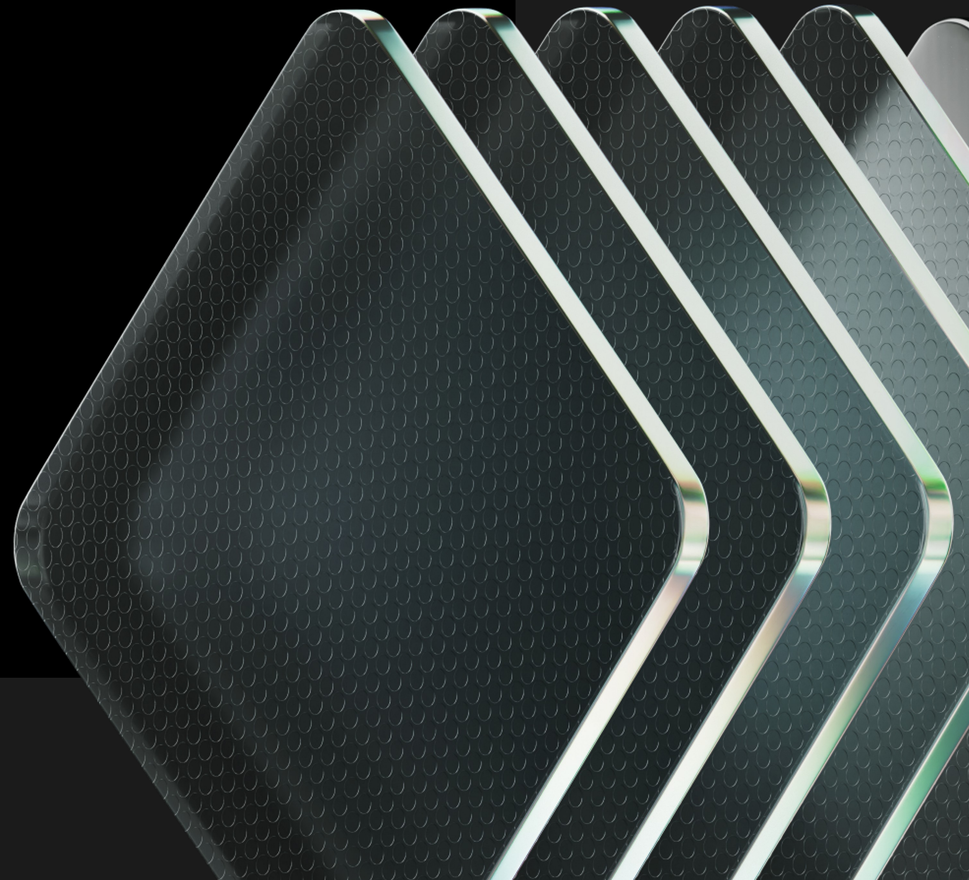
08 FP в std::format

09 lookup(const Key&)

10 Аттрибуты std::thread

01

C++26



C++26

WG21 C++ всё сделала и теперь ожидаем

C++26 ключевые фичи

C++26 ключевые фичи

01

Reflection

^^

[::]

C++26 ключевые фичи

01

Reflection

^^

[::]

02

Contracts

```
pre(x < 0)
post(r: r > 0)
contract_assert(y != 0
```

C++26 ключевые фичи

01

Reflection

^^

[::]

02

Contracts

pre(x < 0)

post(r: r > 0)

contract_assert(y != 0

03

Executors

SIMD

std::hive

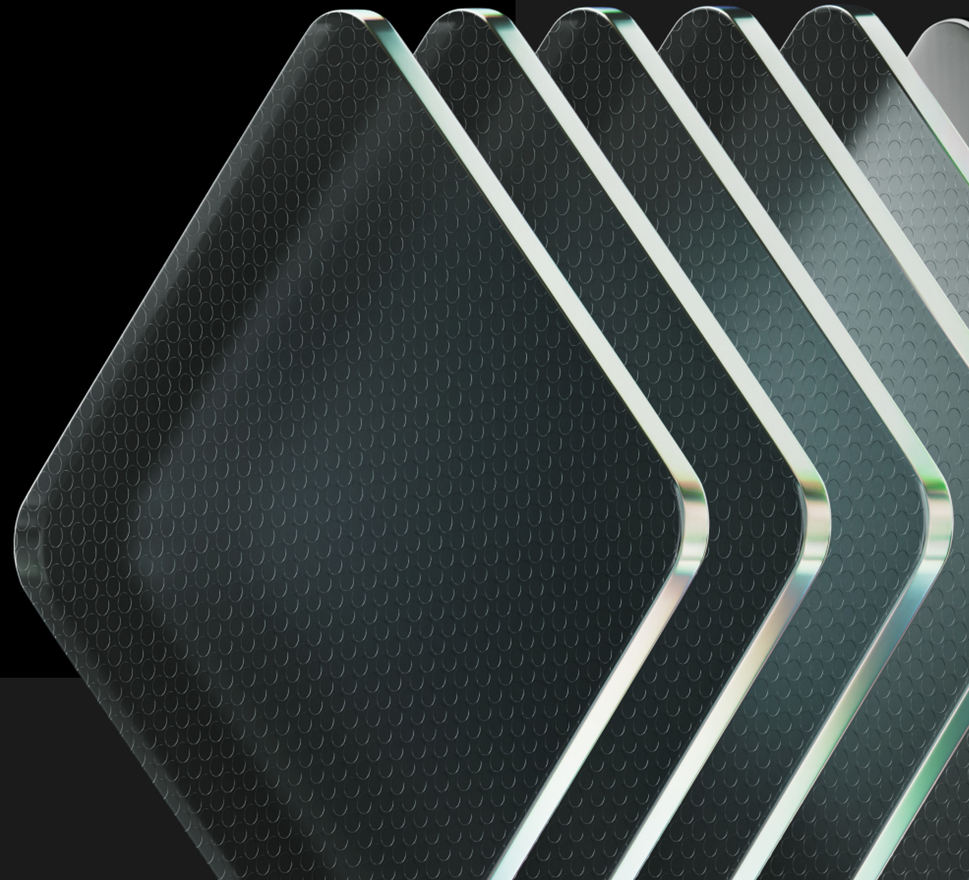
...

C++29

WG21 C++ начинает работу

02

UB & IFNDR



Теперь все UB и IFNDR перечислены

D+a.2.3 **[ub:basic.align.object.alignment]**

Specified in: 6.8.3[ubx:basic.align.object.alignment]

All instances of a type must be created in storage that meets the alignment requirement of that type.

[*Example 1*:

```
struct alignas(4) S {};

void make_misaligned()
{
    alignas(S) char s[sizeof(S) + 1];
    new (&s+1) S();      // undefined behavior, &s+1 will yield a pointer to
                         // a char which is 1 byte away from an address with
                         // an alignment of 4 and so cannot have an alignment of 4.
}
```

— *end example*]

Зачем?

Зачем?



Понятное описание

Зачем?



Понятное описание



Понятные примеры

Зачем?



Понятное описание



Понятные примеры



P3100

- Улучшение диагностики
- Erroneous behavior
- Убирание UB

Один из UB

```
struct X {
    void operator delete(void*) noexcept(false) { throw "oops"; }
};

void f() {
    X* x = new X();
    delete x;                   // undefined behavior
}
```

Один из UB

```
struct X {
    void operator delete(void*) noexcept(false) { throw "oops"; }
};

void f() {
    X* x = new X();
    delete x;                   // undefined behavior
}
```

Один из UB

```
struct X {
    void operator delete(void*) noexcept(false) { throw "oops"; }
};

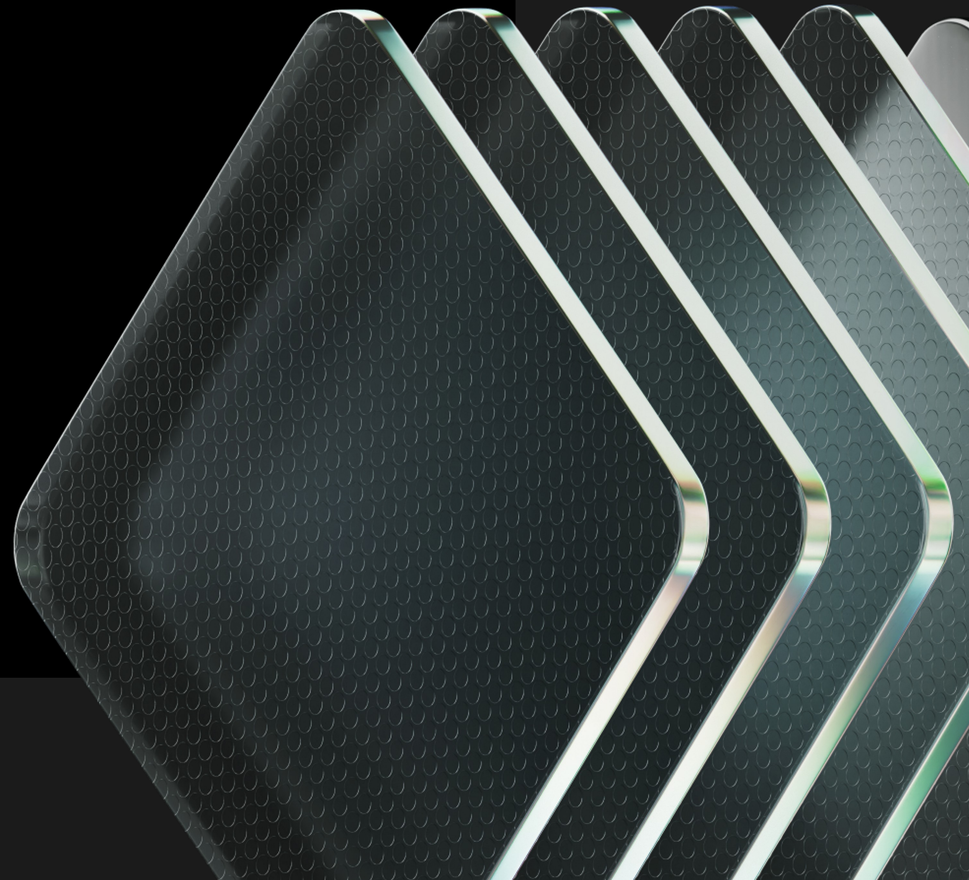
void f() {
    X* x = new X();
    delete x;                   // undefined behavior
}
```

Один из UB

```
struct X {
    void operator delete(void*) noexcept(false)   // Ошибка компиляции
};
```

03

operator++(int) = default;



Boilerplate

```
class Foo {
public:
    constexpr Foo& operator++() {
        ++member_;
        return *this;
    }
    constexpr Foo operator++(int) {
        auto copy{*this};
        this->operator++();
        return copy;
    }
private:
    int member_{0};
};
```

Boilerplate

```
class Foo {
public:
    constexpr Foo& operator++() {
        ++member_;
        return *this;
    }
    constexpr Foo operator++(int) {
        auto copy{*this};
        this->operator++();
        return copy;
    }
private:
    int member_{0};
};
```

No Boilerplate

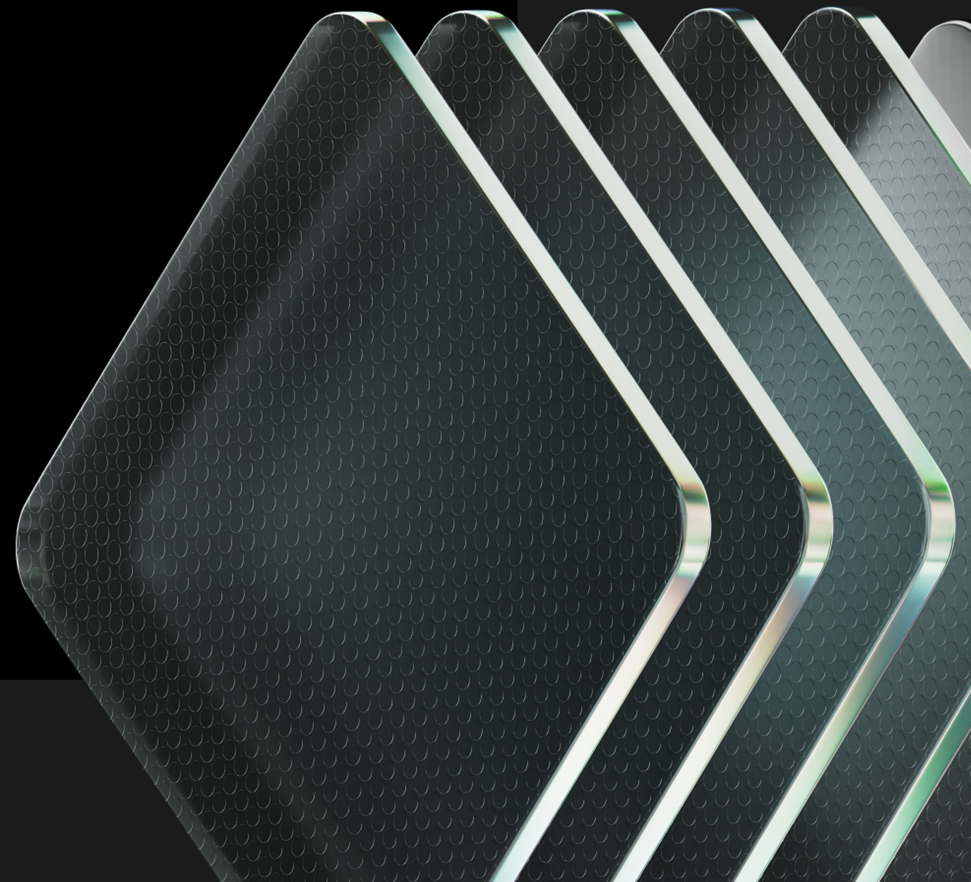
```
class Foo {
public:
    constexpr Foo& operator++() {
        ++member_;
        return *this;
    }
    constexpr Foo operator++(int) = defauit;




private:
    int member_{0};
};
```

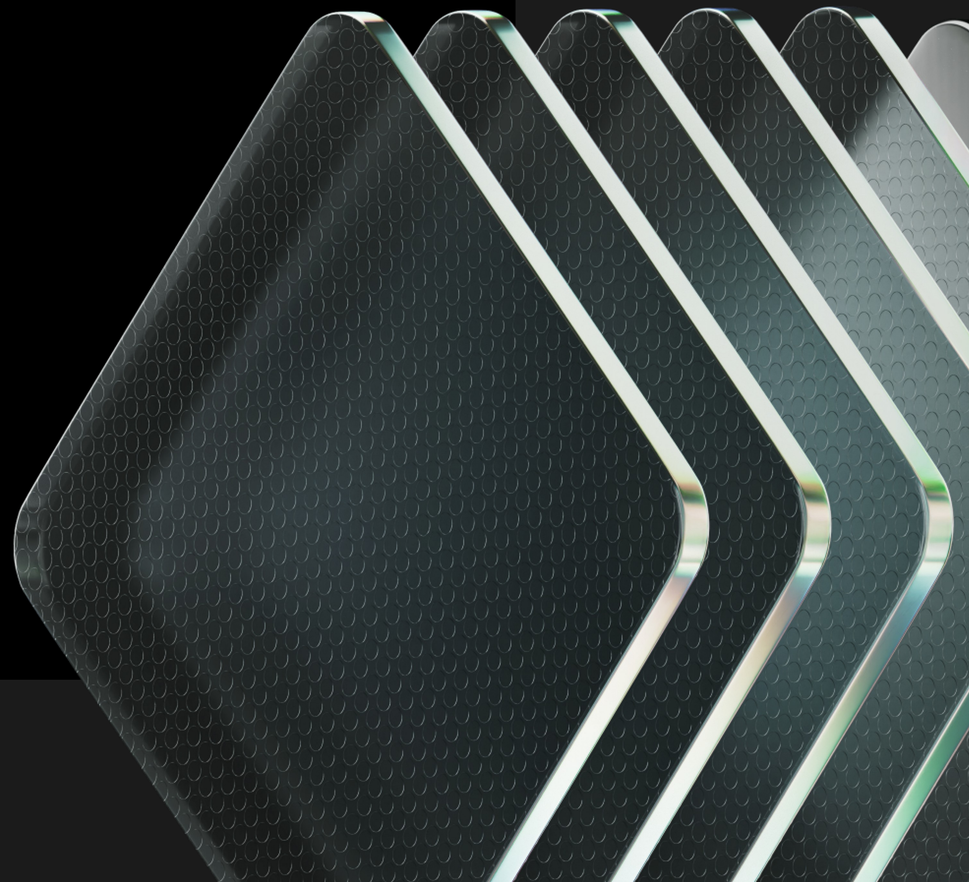
04

std::[u]intptr_t



05

Contracts + virtual



Contracts

```
struct X1 {         void f() pre(a) post(b) {}                  };
struct Y : X1 {     void f() pre(e) post(f) {}                  };

void t() {
    X1 x;
    Y y;

    x.f();                  // asserts a, b
    y.f();                  // asserts e, f
}
```

Contracts + virtual

```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;

    z.f();                  // ???
}
```

Contracts + virtual

```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;

    z.f();                  // asserts g, h
}
```

Contracts + virtual

```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;

    z.f();                  // asserts g, h
    static_cast<Y*>(&z)->f();  // ???
}
```

Contracts + virtual

```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;

    z.f();                  // asserts g, h
    static_cast<Y*>(&z)->f();  // asserts e, g, h, f
}
```

Contracts + virtual

```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;
    X1& x1ref = z;

    void (X1::*pmf)() = &X1::f;
    (x1ref.*pmf)();     // asserts ???
}
```

Contracts + virtual

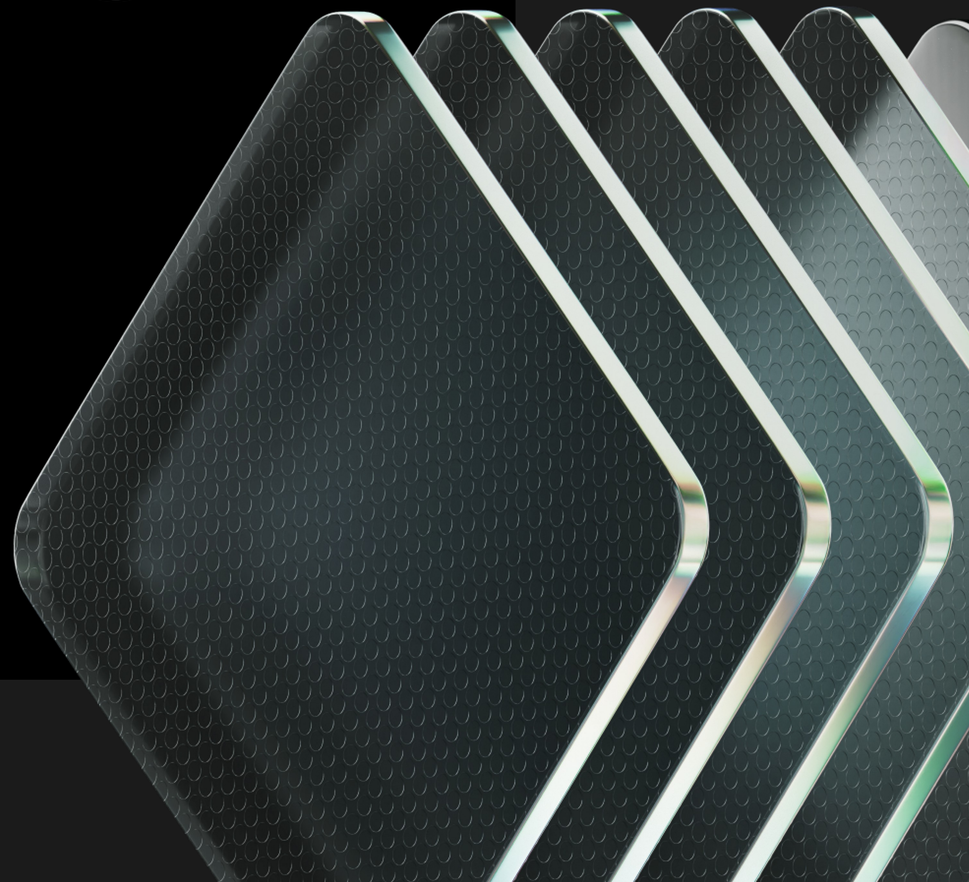
```
struct X1 {        virtual void f() pre(a) post(b) {} };
struct X2 {        virtual void f() pre(c) post(d) {} };
struct Y : X1 {    void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;
    X1& x1ref = z;

    void (X1::*pmf)() = &X1::f;
    (x1ref.*pmf)();     // asserts g,h
}
```

06

Template names indexing



Индексирование шаблонных шаблонов

```
template <typename... T>
struct S {
    using First = T...[0];
};
```

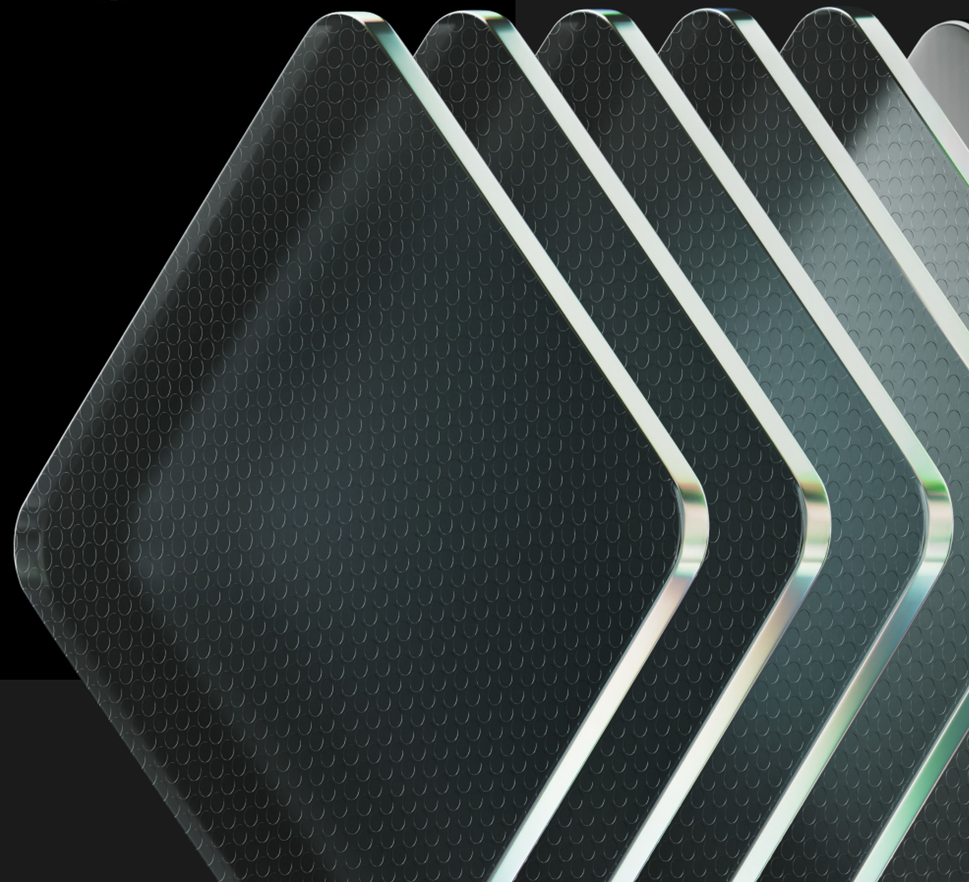
Индексирование шаблонных шаблонов

```
template <typename... T>
struct S {
    using First = T...[0];
};

template < template <typename> typename... TT>
struct S {
    template <typename T>
    using First = TT...[0]<T>;
};
```

07

constant_wrapper - string



std::constant_wrapper

```
template <int I>
void f(std::constant_wrapper<I>) {
    // ...
}

int main() {
    f(std::cw<5>);
}
```

std::constant_wrapper

```
<source>:9:6: error: no matching function for call to 'f(const
std::constant_wrapper<std::_CwFixedValue<int>{5}, int>&)'
    9 |     f(std::cw<5>); // error
      |     ~^~~~~~~~~~~~

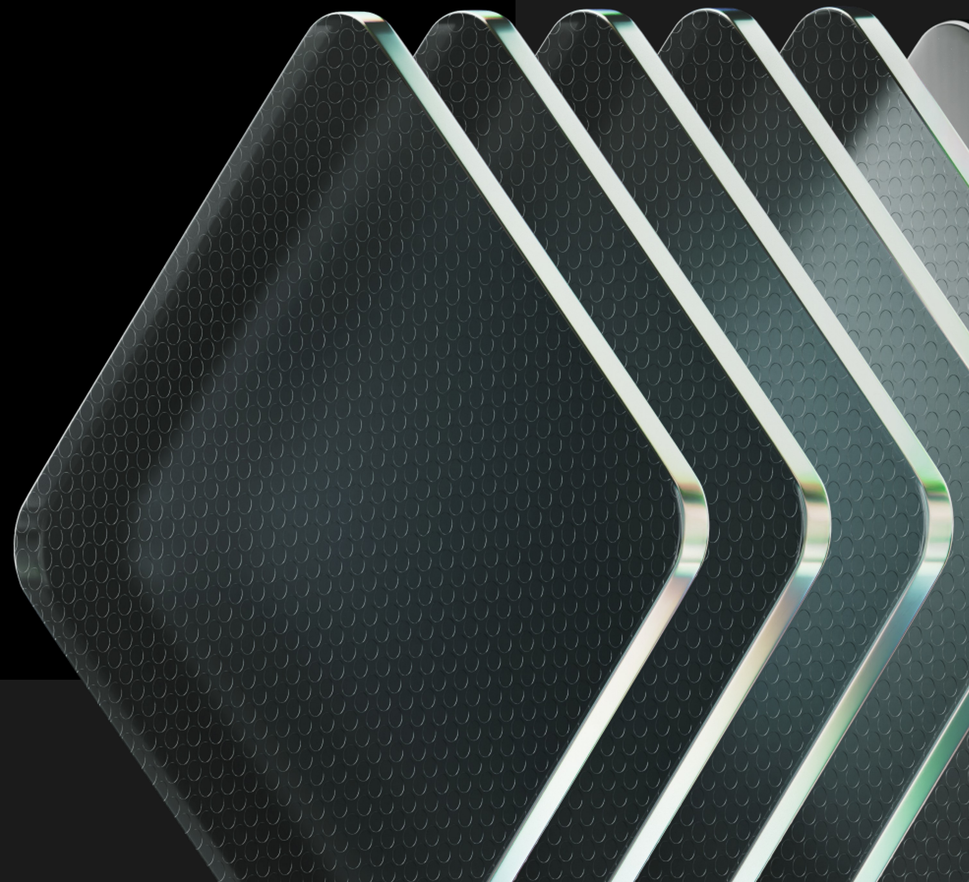
 candidate 1: 'template<int I> void f(std::constant_wrapper<((std::_CwFixedValue<int>)I)>)'
      <source>:4:6:
         4 | auto f(std::constant_wrapper<I>) -> void {
           |      ^
 template argument deduction/substitution failed:
 mismatched types 'int' and 'const std::_CwFixedValue<int>'
```

std::constant_wrapper — string

```
auto a = std::cw<"foo">;
auto b = std::cw<"bar">;
a == b; // error
```

08

FP в std::format



std::format

```
auto s1 = std::format("{}", 120000.0);
```

std::format

```
auto s1 = std::format("{}", 120000.0);  // "120000"
```

std::format

```
auto s1 = std::format("{}", 120000.0);  // "120000"
auto s2 = std::format("{}", 100000.0);
```

std::format

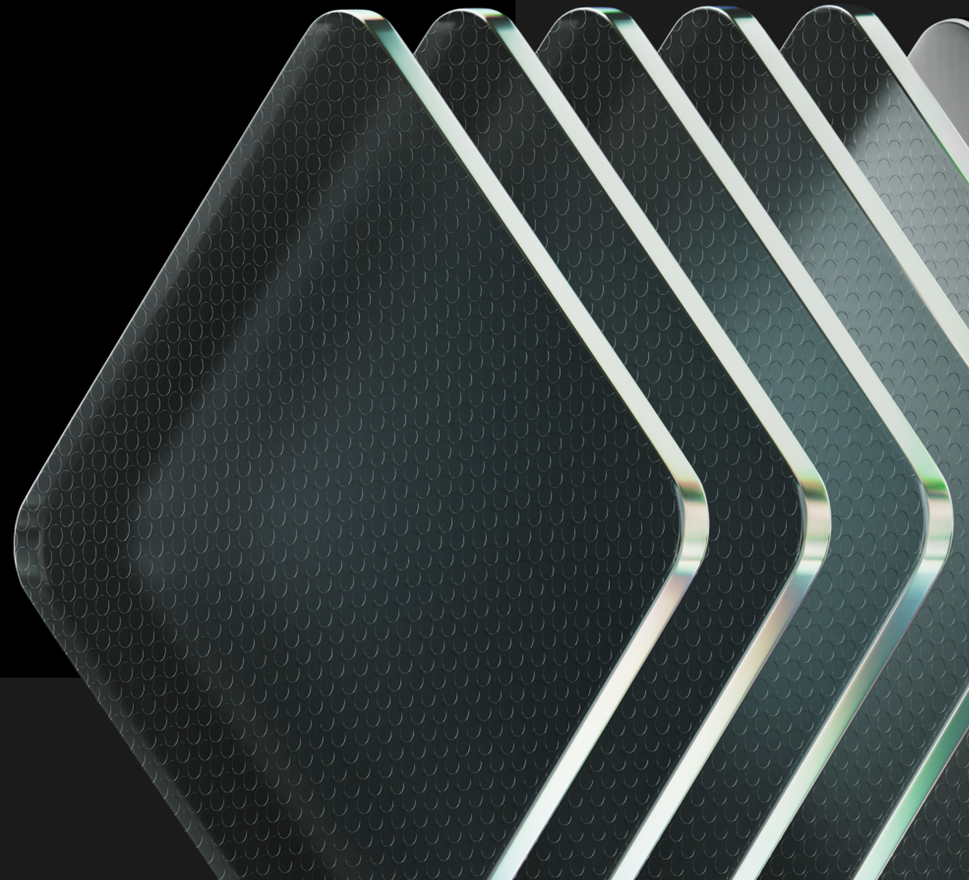
```
auto s1 = std::format("{}", 120000.0);  // "120000"
auto s2 = std::format("{}", 100000.0);  // "1e+05"
```

std::format fixed

```
auto s1 = std::format("{}",   120000.0);  // "120000"
auto s2 = std::format("{}",   100000.0);  // "100000"
auto s3 = std::format("{:L}", 120000.0);  // "120.000"
auto s4 = std::format("{:L}", 100000.0);  // "100.000"
```

09

lookup(const Key&)



find

```
auto do_something(int key, const std::unordered_map<int, int>& cache) {
    int result = 42;
    if (auto it = cache.find(key); it != cache.end()) {
        result = it->second;
    }
    return result;
}
```

find vs lookup

```
auto do_something(int key, const std::unordered_map<int, int>& cache) {
    int result = 42;
    if (auto it = cache.find(key); it != cache.end()) {
        result = it->second;
    }
    return result;
}

auto do_something(int key, const std::unordered_map<int, int>& cache) {
    return cache.lookup(key).value_or(42);
}
```

lookup

```
auto do_something(int key, const std::unordered_map<int, int>& cache) {
    return cache.lookup(key)


        ;
}
```

lookup

```
auto do_something(int key, const std::unordered_map<int, int>& cache) {
    return cache.lookup(key)
                .or_else([&cache] { return cache.lookup(kFallbackValue); })

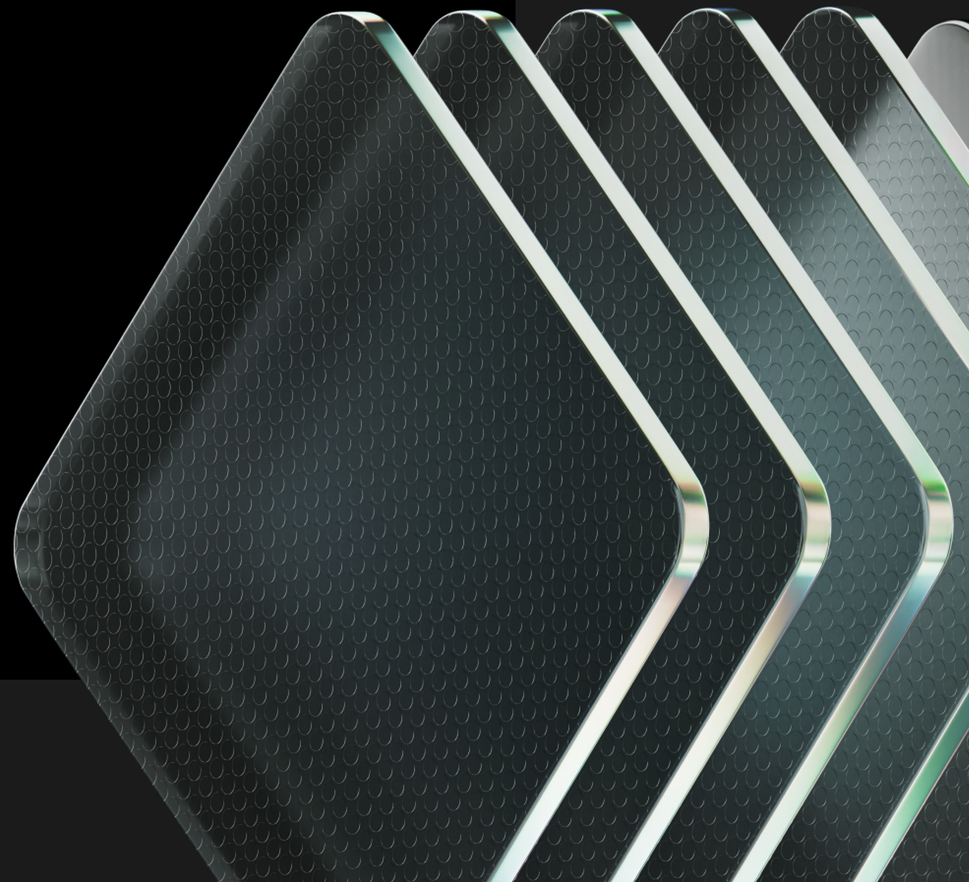
        ;
}
```

lookup

```
auto do_something(int key, const std::unordered_map<int, int>& cache) {
    return cache.lookup(key)
                .or_else([&cache] { return cache.lookup(kFallbackValue); })
                | std::views::transform(...) | ....
        ;
}
```

10

Аттрибуты std::thread



std::[j]thread

```
#include <thread>
void f(int);

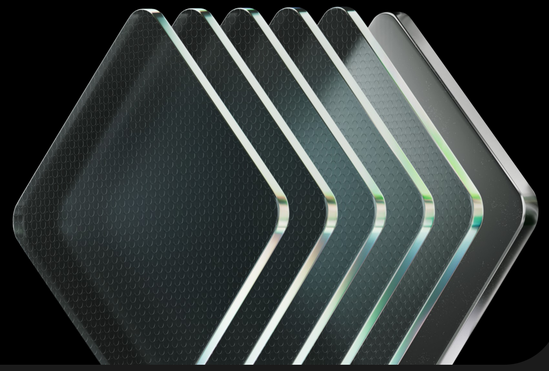
int main() {
    std::thread thread(
        std::thread::name_hint("fs-worker"),
        std::thread::stack_size_hint(512*1024),
        f, 42);
    // ...
    thread.join();
}
```

Яндекс

Спасибо!

Антон Полухин

Техплатформа Городских сервисов Яндекса, руководитель группы разработки общих компонентов



Оцените доклад